



中南大學

CENTRAL SOUTH UNIVERSITY

计算机视觉前沿技术

CSE

计算机学院

线性神经网络

主讲： 沈成超

邮箱： scc.cs@csu.edu.cn

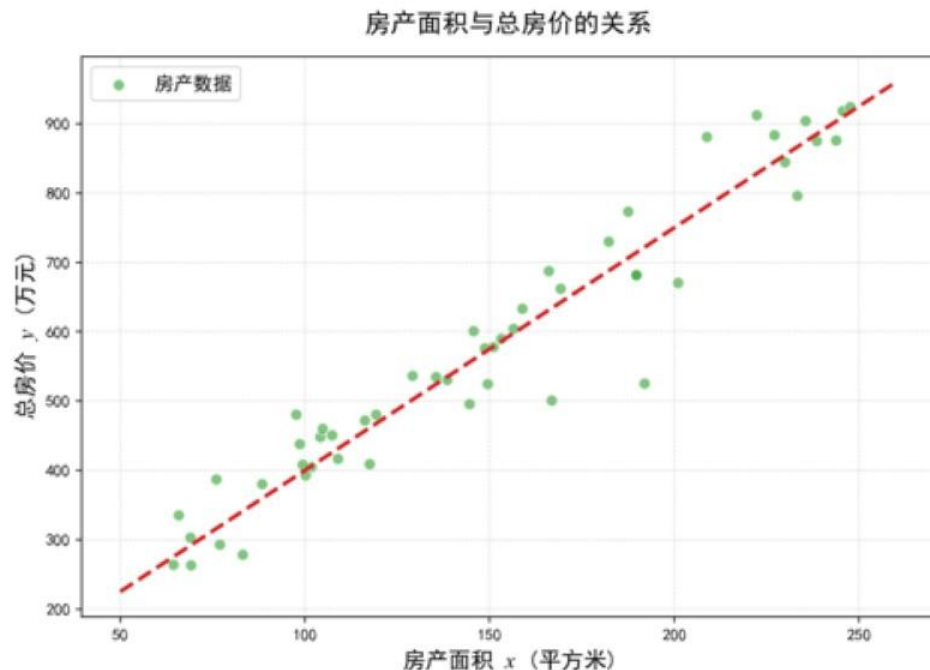
2026.03.13

目录



1. 线性问题
2. 线性神经网络
3. 线性回归
4. 线性分类
5. 梯度下降
6. 课程作业

1. 线性问题：线性回归



线性回归：

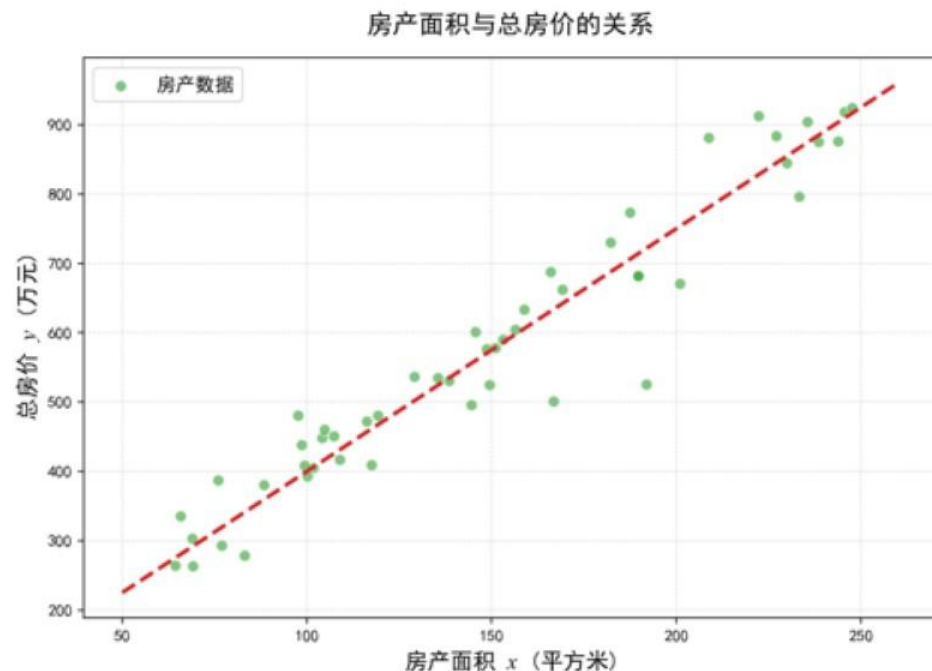
给定 x ，如何预测 y ，其中 x 和 y 均为连续值

问题定义：

寻找一个线性函数，用来描述一个或多个自变量（特征 x_i ）与一个连续的因变量（目标值 y_i ）之间的关系

$$\hat{y}_i = wx_i + b$$

1. 线性问题：线性回归



优化目标：最小化的是误差平方和

$$L(w, b) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (wx_i + b))^2$$

求解方法：求函数极小值

$$\frac{\partial L}{\partial w} = -2 \sum_{i=1}^n x_i (y_i - wx_i - b) = 0$$

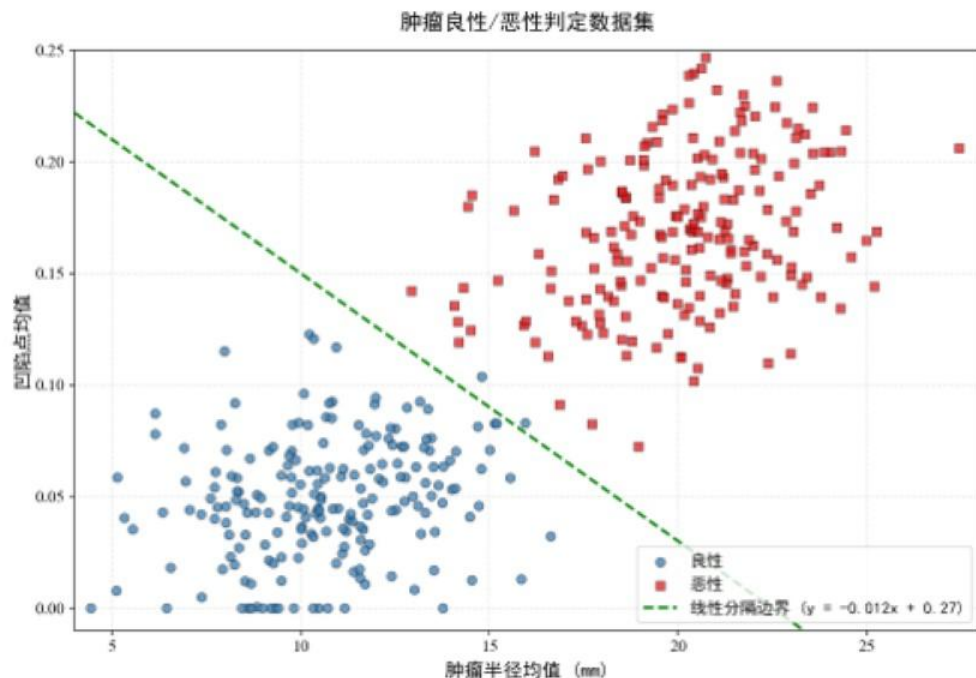
$$\frac{\partial L}{\partial b} = -2 \sum_{i=1}^n (y_i - wx_i - b) = 0$$

矩阵求解： $\hat{Y} = X\beta$

目标函数： $L(\beta) = (Y - X\beta)^T (Y - X\beta)$

解析解： $\beta = (X^T X)^{-1} X^T Y$

1. 线性问题：线性分类



- **输入 (特征X)**：与肿瘤相关的多维度数据，如医学影像特征（大小、形状、密度等）、病理学指标或临床体征。
- **输出 (标签Y)**：仅包含两个互斥的类别，通常定义为**良性**（常标记为0）和**恶性**（常标记为1）。 → **离散值预测**
- **目标**：寻找或训练一个映射函数 $f(X) = Y$ ，使得模型能够根据已知特征，准确预测未知肿瘤属于良性还是恶性。

2. 线性神经网络：定义

- 给予 n 维输入, $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$

- 线性方法有 n 个权重和偏差:

$$\mathbf{w} = [w_1, w_2, \dots, w_n]^T, b$$

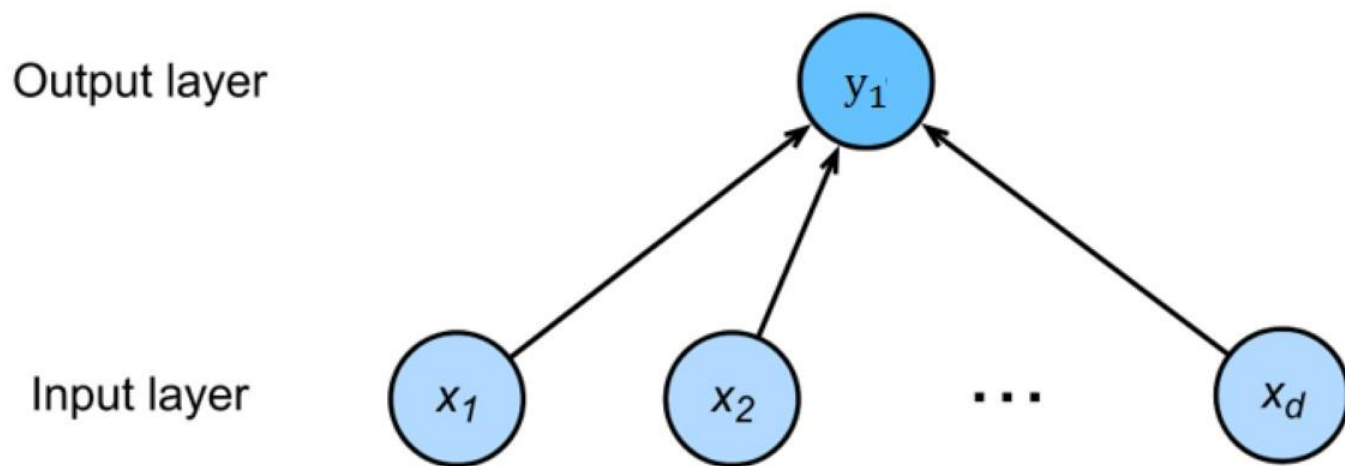
- 输出是输入的加权总和:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

矢量化版本:

$$y = \langle \mathbf{w}, \mathbf{x} \rangle + b$$

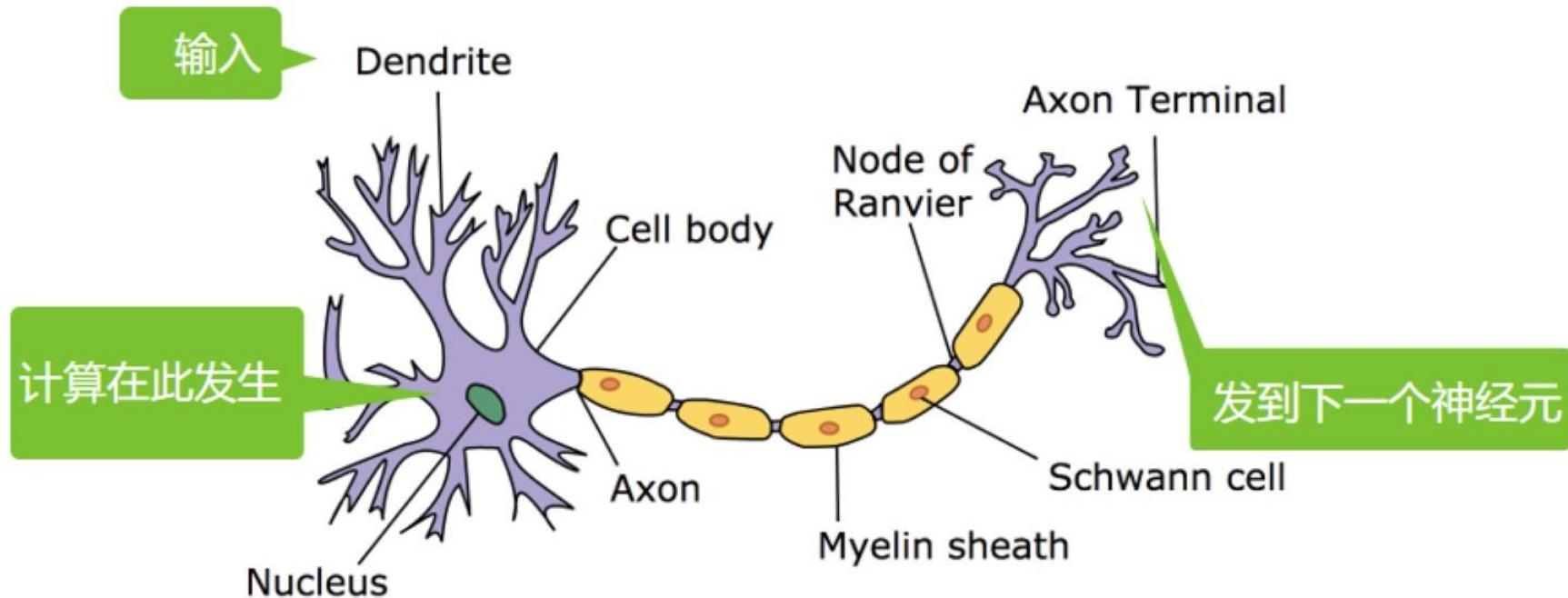
2. 线性神经网络：定义



我们可以堆叠多个层来获得深层神经网络。

2. 线性神经网络：定义

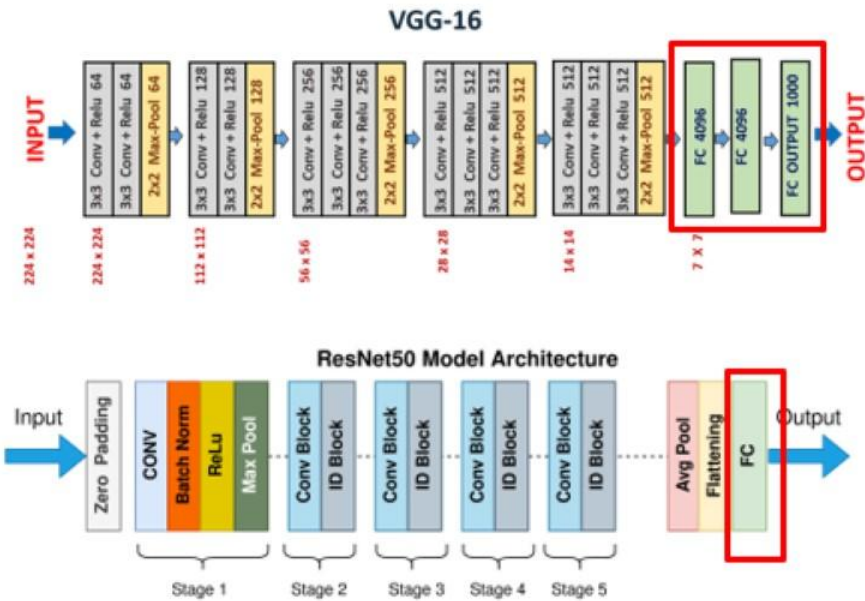
真实的神经元



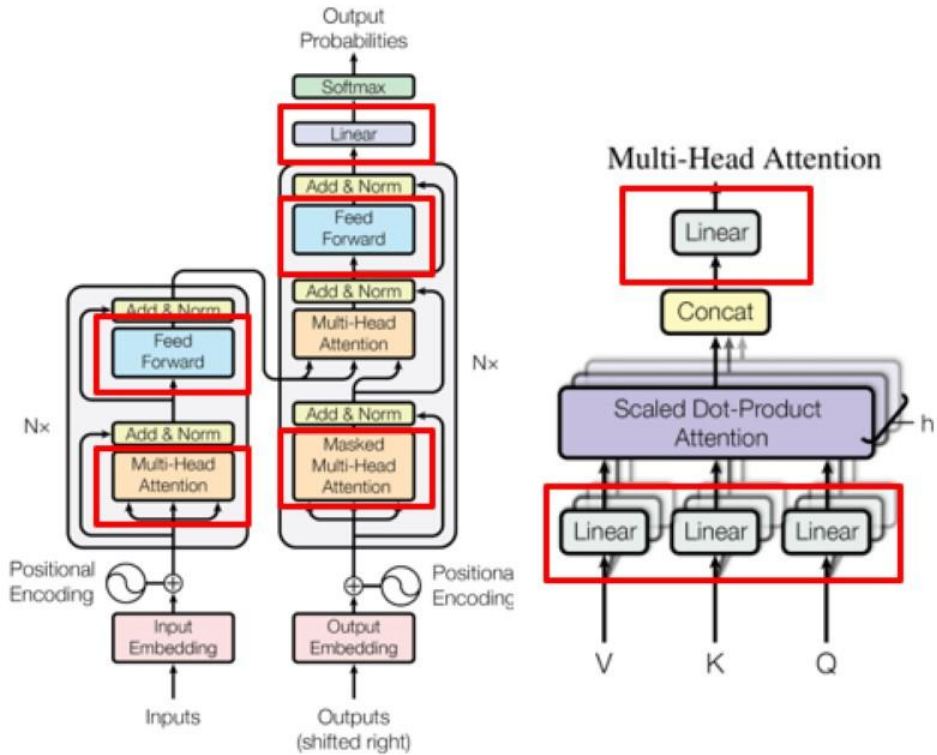
2. 线性神经网络：重要性

线性模块是现有神经网络、Transformer的核心模块

• CNN



• Transformer



2. 线性神经网络：重要性

线性模块 → 线性代数：矩阵 → 解析几何 → 几何意义强、可解释性强、简单

旋转、伸缩的复合变换

投影、相似性

- 线性代数的本质 - 04 - 矩阵乘法与线性变换复合
<https://www.bilibili.com/video/BV1ms41167u9>
- 线性代数的本质 - 04补充 - 三维空间中的线性变换
<https://www.bilibili.com/video/BV1Ss41167dG>

3. 线性回归：训练数据集

- 收集多个数据点以训练模型参数，称为**训练数据集**
- 训练数据集 越大越好
- 假设有 n 个数据

$$\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_n]^T \quad \mathbf{z} = [z_0, z_1, \dots, z_n]^T$$

3. 线性回归：目标函数 / 损失函数

- 训练损失

$$\ell(\mathbf{X}, \mathbf{z}, \mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (z_i - \langle \mathbf{x}_i, \mathbf{w} \rangle - b)^2 = \frac{1}{n} \| \mathbf{z} - \mathbf{X}\mathbf{w} - b \|^2$$

- 最小化学习参数的损失

$$\mathbf{w}^*, \mathbf{b}^* = \operatorname{argmin}_{\mathbf{w}, b} \ell(\mathbf{X}, \mathbf{z}, \mathbf{w}, b)$$

3. 线性回归：最优解析解（最小二乘法）

- 将偏差添加到权重中 $\mathbf{X} \leftarrow [\mathbf{X}, \mathbf{1}] \mathbf{w} \leftarrow \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{n} \|\mathbf{z} - \mathbf{X}\mathbf{w}\|^2 \qquad \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{2}{n} (\mathbf{z} - \mathbf{X}\mathbf{w})^T \mathbf{X}$$

- 损失是凸性的，因此最优解满足：

$$\frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{z}, \mathbf{w}) = 0$$

$$\Leftrightarrow \frac{2}{n} (\mathbf{z} - \mathbf{X}\mathbf{w})^T \mathbf{X} = 0$$

$$\Leftrightarrow \mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X} \mathbf{z}$$

4. 线性分类

- 回归估计连续值
- 分类预测离散类别

MNIST: 对手写数字进行分类 (10 类)

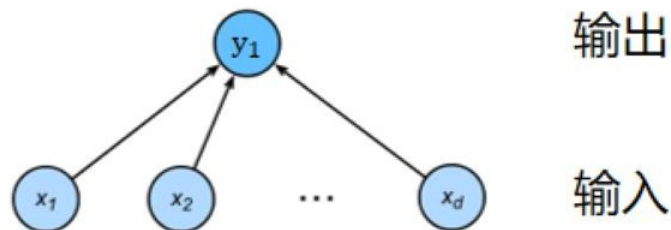


ImageNet: 对自然对象进行分类 (1000 类)

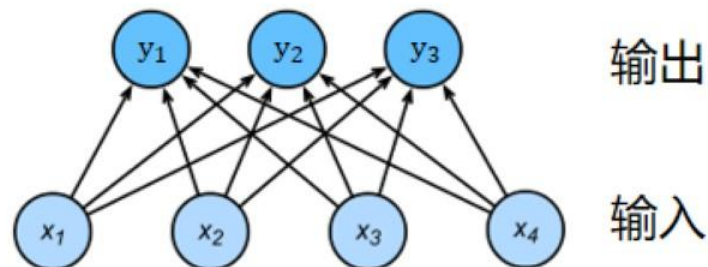


4. 线性分类

- 单个连续数值输出



- 输出每个类别的置信度分数



- 预测类别

$$\operatorname{argmax}_i(y_1, y_2, y_3)$$

argmax 不可求导
定义一个损失函数

4. 线性分类：目标函数 / 损失函数

$$S = \text{softmax}([y_1, y_2, \dots, y_n]^T) = \left[\frac{e^{y_1}}{\sum_{i=1}^n e^{y_i}}, \frac{e^{y_2}}{\sum_{i=1}^n e^{y_i}}, \dots, \frac{e^{y_n}}{\sum_{i=1}^n e^{y_i}} \right]$$

$$S_i = \frac{e^{y_i}}{\sum_{k=1}^n e^{y_k}}$$

- 用 exp 获得大于0的值
- 除以总和以获得**概率分布**

4. 线性分类：目标函数 / 损失函数

是否用平方损失？

- 对 y 进行一位有效编码 (One-hot encoding)

$$\mathbf{z} = [c_1, c_2, \dots, c_n]^T, c_i = \begin{cases} 1 & \text{if } i = z \\ 0 & \text{otherwise} \end{cases}$$

- 我们需要 softmax 结果接近 $\mathbf{z}$
- 用平方损失： $\mathbf{z} = [0, 0, 1]$

softmax 结果	损失
$[0.3, 0, 0.7]$	0.18
$[0.17, 0.17, 0.66]$	0.173

4. 线性分类：目标函数 / 损失函数

交叉熵 (Cross Entropy)

- 一位有效编码与 softmax 运算结果都为概率分布
- 交叉熵通常用于比较概率分布

$$H(\mathbf{y}, S) = - \sum_{i=1}^n y_i \log(S_i)$$

如果 $y = i$, 那么

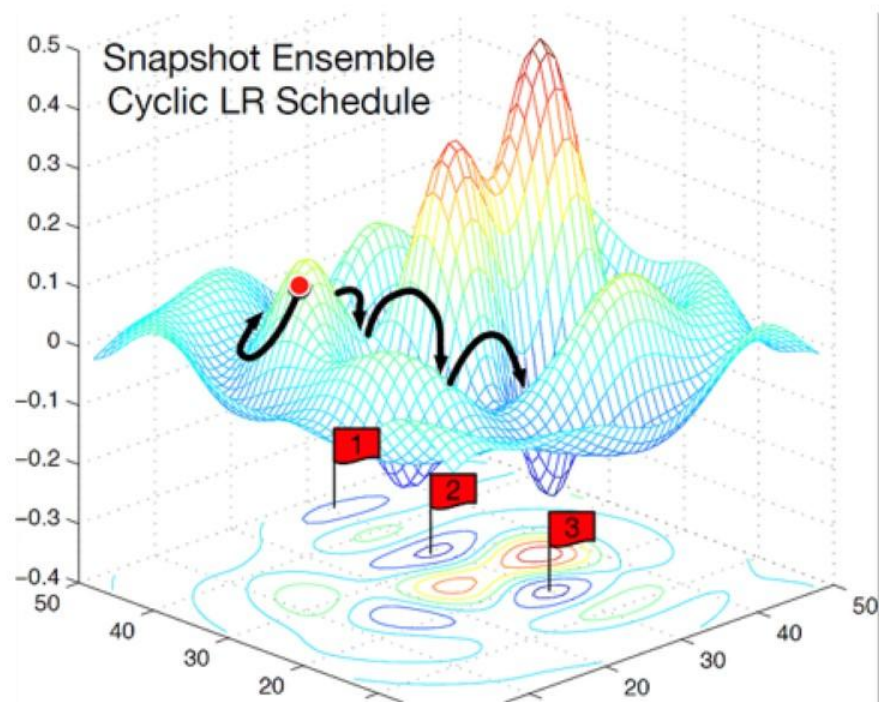
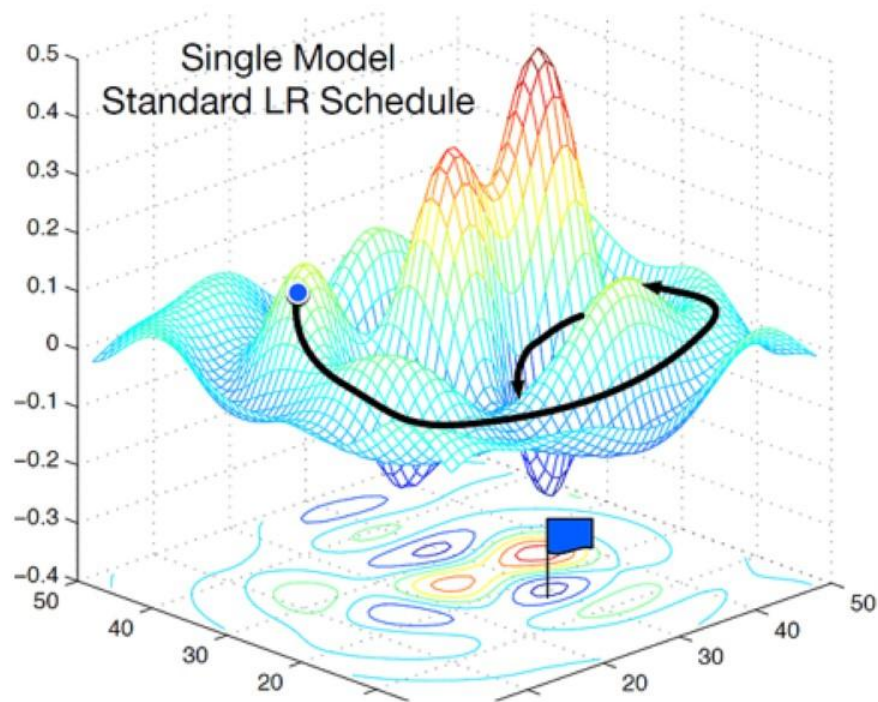
$$H(\mathbf{y}, \hat{\mathbf{y}}) = -\log(S_i)$$

4. 线性分类：线性回归 vs 线性分类

问题	线性回归 (回归问题)	线性分类 (分类问题)
模型	$\langle \mathbf{w}, \mathbf{x} \rangle + b$ $\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}$	$\text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$ $\mathbf{W} \in \mathbb{R}^{k \times n}, \mathbf{b} \in \mathbb{R}^k$
损失	平方损失	交叉熵
最优解	解析解	数值解
预测目标	连续值	离散值

5. 梯度下降：损失函数优化

如何快速下山？ → 从坡度最陡的地方往下走（贪婪算法） → 梯度下降

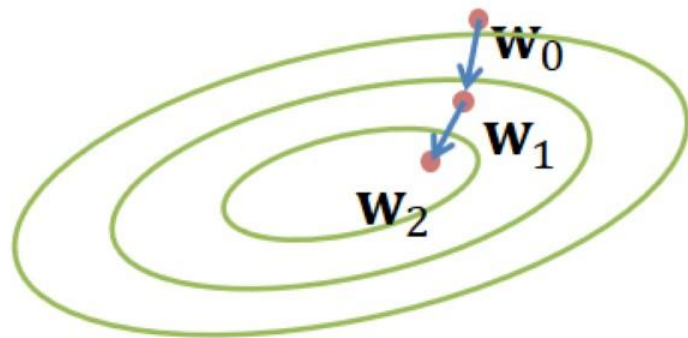


5. 梯度下降：损失函数优化

- 选择一个起点 $\mathbf{w}_0$
- 重复更新权重 $t = 1, 2, 3$

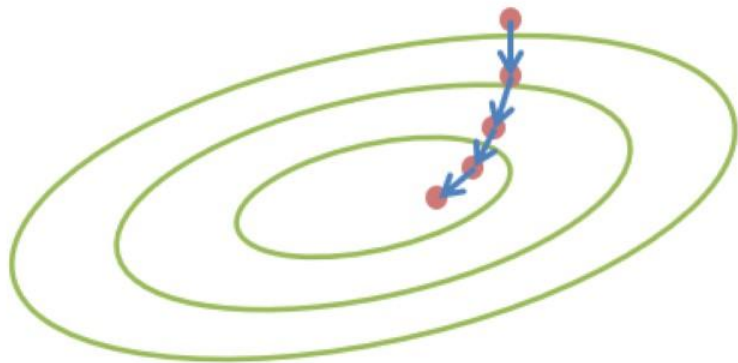
$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$$

- 梯度 $\frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$ ：更新权重的方向
- 学习率 η ：一个超参数指定 每梯度的步长

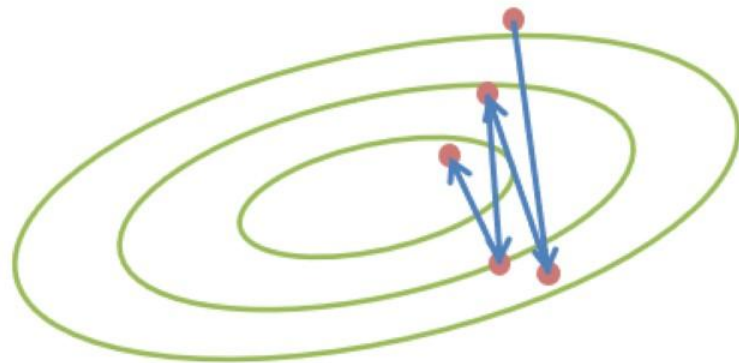


5. 梯度下降：学习率设置

不要太小



不要太大



5. 梯度下降：小批量(mini-batch)随机梯度下降 (SGD)

- 计算整个训练数据的梯度太昂贵了
 - 内存无法容纳
 - 计算时间过长
- 解决方案： 随机抽样b个样本 $i_1, i_2, \dots, i_b$ 来估算损失

$$\frac{1}{b} \sum_{i \in I_b} \ell(\mathbf{x}_i, y_i, \mathbf{w})$$

b 是批量大小 (batch size) , 另一个重要的超参数

5. 梯度下降：梯度计算

$$H(\mathbf{y}, S) = -\log(S_i) \qquad S_i = \frac{e^{y_i}}{\sum_{k=1}^n e^{y_k}}$$

$$\frac{\partial H}{\partial w_j} = \frac{\partial H}{\partial S_i} \frac{\partial S_i}{\partial y_j} \frac{\partial y_j}{\partial w_j} \qquad \frac{\partial H}{\partial S_i} = -\frac{1}{S_i} \qquad \frac{\partial y_j}{\partial w_j} = x_j$$

当 $j = i$ 时

$$\frac{\partial S_i}{\partial y_i} = \frac{e^{y_i}(\sum_{k=1}^n e^{y_k}) - e^{y_i}e^{y_i}}{(\sum_{k=1}^n e^{y_k})^2} = \frac{e^{y_i}}{\sum_{k=1}^n e^{y_k}} - \left(\frac{e^{y_i}}{\sum_{k=1}^n e^{y_k}}\right)^2 = S_i - S_i^2$$

 $\frac{\partial H}{\partial w} = \frac{\partial H}{\partial S_i} \frac{\partial S_i}{\partial y_i} \frac{\partial y_i}{\partial w_i} = (S_i - 1)x_i$

当 $j \neq i$ 时

$$\frac{\partial S_i}{\partial y_j} = \frac{0 \times \sum_{k=1}^n e^{y_k} - e^{y_j}e^{y_i}}{(\sum_{k=1}^n e^{y_k})^2} = -\frac{e^{y_i}}{\sum_{k=1}^n e^{y_k}} \frac{e^{y_j}}{\sum_{k=1}^n e^{y_k}} = -S_i S_j$$

 $\frac{\partial H}{\partial w_j} = \frac{\partial H}{\partial S_i} \frac{\partial S_i}{\partial y_j} \frac{\partial y_j}{\partial w_j} = -S_j x_j$

5. 梯度下降：训练过程

1. 读取数据集 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, $y = [y_1, y_2, \dots, y_n]^T$
2. 初始化线性模型 $y = \langle \mathbf{w}, \mathbf{x} \rangle + b$
3. 选择损失函数 ℓ 、学习率 η 、批大小
4. 训练循环（例如：数据集上迭代100 epochs/遍）

读取一个批次数据 $b_i = [(x_k, y_k), \dots]$

批数据输入模型： $y = \langle \mathbf{w}, b_i \rangle + b$

计算损失函数，并求梯度 $\frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$

更新模型参数： $\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$

5. 保存最终得到的模型权重 $\mathbf{w}$, b

6. 课程作业

- 根据给定数据文件（2维数据），进行线性二分类（softmax + 交叉熵 + 梯度下降）
- 用Numpy完成，不能使用PyTorch、Tensorflow等自动计算梯度的python库；
- 记录并可视化训练过程的损失函数变化（10 epochs）；
- 可视化分类边界，展示不同训练阶段分类边界的变化过程（**可选，额外加分**）；
- 可以用AI辅助，同时提供AI代码（学习）+自写代码（不看AI代码后自己）